FUNDAMENTALS OF AL - ML

MOVIE RECOMMENDATION PROGRAM

VIT BHOPAL UNIVERSITY

Submitted By: *Adit Sharma* **Submitted To:** BANDLA PAVAN BABU

★AI - ML PROJECT REPORT FILE

Project : AI BASED MOVIE RECOMENDATION

Aim: To get the recommendation of movies according to the Category and movie name

Cover Page

Project Title: AI Movie Recommendation
Submitted By: *Adit Sharma*

Reg no. 25BCE10716
Program & Branch: B. Tech – Computer Science (core)
Course Name: Fundamentals in AL ML
Institution: VIT BHOPAL UNIVERSITY
Academic Year: 2025–26
Submitted To: BANDLA PAVAN BABU

• INTRODUCTION :

In today's digital world, people are overwhelmed with a massive number of movies available across various platforms. Choosing what to watch has become confusing and time-consuming. A movie recommendation system helps users discover movies based on their interests, preferences, and past viewing behaviors.

This project focuses on building an interactive, intelligent movie recommendation program using AI/ML concepts. Unlike basic systems, this updated program analyzes multiple contextual data points (Genre, Director, Cast, Language, Year) and even forgives user typos. The goal is to make the process of selecting movies easier, faster, highly interactive, and remarkably personalized for users.

• Problem Statement :

With the rapid growth of online streaming platforms, users often struggle to find movies that match their specific tastes. They either spend too much time searching or end up watching something they don't enjoy. Furthermore, users often misspell movie names when searching, leading to frustrating "no results found" errors.

The problem is to design a system that can:

- Understand complex user preferences based on multiple movie attributes.
- Intelligently auto-correct user input errors (typos).
- Filter relevant movies from a large dataset interactively.
- Provide accurate and personalized recommendations using Natural Language Processing (NLP) techniques

Functional Requirements

The updated movie recommendation system must perform the following functions:

User Input & Interaction:

- **Provide a highly interactive, menu-driven terminal UI.**
- **Accept movie names to find similar matches, accounting for spelling mistakes.**

- **Allow users to browse specifically by categories like Genre, Director, Language, Year, or Cast.**

Data Handling:

- **Load and manage a dataset of movies (movies.csv).**
- **Preprocess data to handle missing values automatically.**
- **Combine multiple text features into a single "AI Context" string for better mathematical matching.**

Recommendation Logic:

- **Calculate recommendations using TF-IDF (Term Frequency-Inverse Document Frequency) and Cosine Similarity.**
- **Utilize partial string matching for the category browsing feature.**

•Non-Functional Requirements:

These requirements focus on how well the system works:

- **Performance:** The model must be pre-built upon running the script so that subsequent searches and recommendations generate instantly without delay.
- **Usability:** The interface should utilize an intuitive terminal UI (questionary library) with clear menus and arrow-key navigation.
- **Robustness:** The system must not crash if the user types a movie name incorrectly; it should attempt to find the closest match.
- **Maintainability:** The code must be Object-Oriented (using a MovieRecommender class) for clean structure and easy future updates.

•System Architecture:

The system follows a streamlined architecture tailored for this ML project:

1. **User Interface (UI) Layer:** Powered by the questionary library. This is where the user interacts with the system using visual terminal prompts, dropdowns, and text inputs.
2. **Processing Layer:** The core brain of the system.
    - Uses difflib for typo correction and input matching.
    - Uses TfidfVectorizer to convert text into mathematical vectors.
    - Applies cosine_similarity to find the closest relational distances between movies.

3. **Data Layer:** Stores the movie dataset. Uses pandas to read the CSV, clean column names, and merge features (Genre, Director, Cast, etc.) into a cohesive data frame.
4. **Output Layer:** Formats the final arrays into clean, readable terminal text displaying top suggestions with contextual details.

• FLOW CHART OF WORKING:

 Start 2. **Initialize Application:** Load dataset (movies.csv), fill missing values, build TF-IDF matrix.

 Display Interactive Menu: * *Option A:* Find similar movies.

- *Option B:* Browse by category.
- *Option C:* Exit.

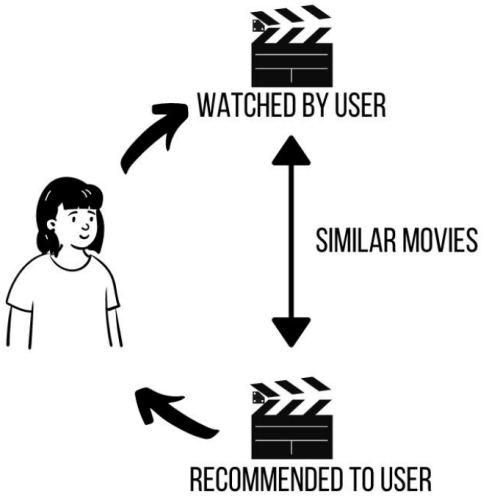
 Process Input (Based on selection):

- *If Option A:* Take movie name -> Check for typos (difflib) -> Calculate Cosine Similarity -> Sort matches.
- *If Option B:* Select Category -> Take search term -> Apply partial text matching (pandas filtering).

 Generate & Display Results: Print formatted list of top 5 to 10 movies.

 Loop/End: Prompt user to return to the main menu or exit the program.

```
        Start
          |
          v
 Take User Input
(Genre / Movie)
          |
          v
 Load Movie Dataset
          |
          v
 Apply Recommendation Logic
(Filter / Similarity Check)
          |
          v
 Generate Recommendations
          |
          v
  Display Results
          |
          v
        End
```



• Implementation Details :

1. Programming Language & Tools

- **Language: Python**
- **Libraries Used:**
  - **Pandas: For loading, managing, and structuring the dataset.**
  - **Scikit-learn: For TfidfVectorizer (text processing) and cosine_similarity (calculating match percentages).**
  - **Difflib: For built-in typo forgiveness and string matching.**
  - **Questionary: For building beautiful, interactive terminal prompts.**

2. Data Preprocessing Before making recommendations, the system cleans the data:

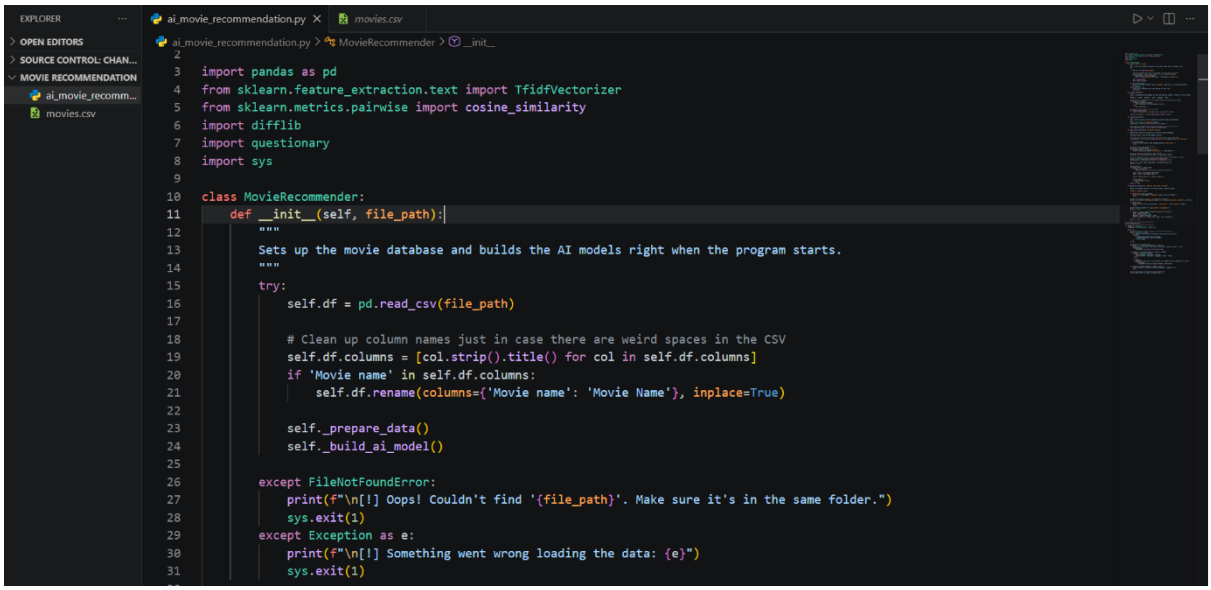
- **Standardizes column names and strips whitespace.**
- **Replaces empty cells (NaN) with blank strings to prevent math errors.**

- **Combines Genre, Director, Cast, Language, and Year into a single, unified ai_context column.**

3. Recommendation Technique A Content-Based Filtering approach using advanced NLP is used:

- **Instead of simple word counts, TF-IDF Vectorization is used to weigh how unique and important certain words (like directors or niche genres) are across the whole dataset.**
- **Cosine Similarity calculates the angle between these vectors to determine how closely related two movies are on a scale of 0 to 1.**

•Coding Part:



```python
class MovieRecommender:

    def _prepare_data(self):
        """
        Fills in missing data and combines all text into one big 'context' string for the AI to read.
        """
        features = ['Genre', 'Director', 'Cast', 'Language', 'Year']

        # Replace empty spreadsheet cells with empty strings so the math doesn't break
        for feature in features:
            if feature in self.df.columns:
                self.df[feature] = self.df[feature].fillna('')
            else:
                self.df[feature] = ''

        # Combine the text features into one column
        def combine_features(row):
            return f"{row['Genre']} {row['Director']} {row['Cast']}".lower()

        self.df['ai_context'] = self.df.apply(combine_features, axis=1)

    def _build_ai_model(self):
        """
        Uses TF-IDF to figure out which keywords are actually unique and important.
        """
        tfidf = TfidfVectorizer(stop_words='english')
        tfidf_matrix = tfidf.fit_transform(self.df['ai_context'])

        # Calculate how mathematically similar every movie is to every other movie
        self.similarity_scores = cosine_similarity(tfidf_matrix)

```

```python
class MovieRecommender:

    def find_similar_movies(self, user_movie, top_n=5):
        """
        Finds movies based on AI similarity, with built-in typo forgiveness.
        """
        all_movie_titles = self.df['Movie Name'].tolist()

        # AI Typo Correction: Find the closest real movie title (requires 60% match)
        close_matches = difflib.get_close_matches(user_movie, all_movie_titles, n=1, cutoff=0.6)

        if not close_matches:
            print(f"\n[?] We couldn't find anything matching '{user_movie}'.")
            return

        # Grab the exact title the AI decided on
        best_match = close_matches[0]
        if best_match.lower() != user_movie.lower():
            print(f"\n(AI Auto-Corrected '{user_movie}' to '{best_match}')")

        # Look up the movie's ID number in our dataframe
        movie_idx = self.df[self.df['Movie Name'] == best_match].index[0]

        # Get the similarity scores for this specific movie and sort them highest to lowest
        scores = list(enumerate(self.similarity_scores[movie_idx]))
        sorted_scores = sorted(scores, reverse=True, key=lambda x: x[1])

        print(f"\n🎬 If you liked '{best_match}', you should check out:")
        print("-" * 45)

        results_shown = 0
        for index, score in sorted_scores:
            if index == movie_idx:
                continue # Skip the movie the user actually searched for
```

```python
class MovieRecommender:
    def find_similar_movies(self, user_movie, top_n=5):

            title = self.df.iloc[index]['Movie Name']
            genre = self.df.iloc[index]['Genre']

            print(f"{results_shown + 1}. {title} ({genre})")

            results_shown += 1
            if results_shown >= top_n:
                break
        print("-" * 45)

    def browse_by_category(self, category, search_term, top_n=10):
        """
        Filters the database based on a text search within a specific column.
        """
        category = category.title()

        if category not in self.df.columns:
            print(f"\n[!] The category '{category}' doesn't exist in the data.")
            return

        # Search the column for partial, case-insensitive matches
        matches = self.df[self.df[category].astype(str).str.contains(search_term, case=False, na=False)]

        if matches.empty:
            print(f"\n[?] No movies found matching '{search_term}' in the {category} category.")
            return

        print(f"\n📂 Top results for '{search_term}' in {category}:")
        print("-" * 45)

        for count, (_, row) in enumerate(matches.head(top_n).iterrows()):
```

```python
class MovieRecommender:
    def browse_by_category(self, category, search_term, top_n=10):

        for count, (_, row) in enumerate(matches.head(top_n).iterrows()):
            title = row['Movie Name']
            year = row.get('Year', 'N/A')
            director = row.get('Director', 'N/A')
            print(f"{count + 1}. {title} (Year: {year} | Dir: {director})")

        print("-" * 45)

# ========================================
# MAIN INTERACTIVE APP
# ========================================
if __name__ == "__main__":
    recommender = MovieRecommender('movies.csv')

    while True:
        # Use Questionary to create a beautiful interactive terminal menu
        action = questionary.select(
            "🍿 Welcome to the AI Movie Recommender! What would you like to do?",
            choices=[
                "🔍 Find movies similar to a favorite",
                "📂 Browse movies by a specific category",
                "❌ Exit Program"
            ]
        ).ask()

        # Handle the user's menu choice
        if action == "🔍 Find movies similar to a favorite":
            movie_input = questionary.text("Enter a movie name (typos are okay!):").ask()
            if movie_input:
                recommender.find_similar_movies(movie_input)
```

```python
        # Use Questionary to create a beautiful interactive terminal menu
        action = questionary.select(
            "🍿 Welcome to the AI Movie Recommender! What would you like to do?",
            choices=[
                "🔍 Find movies similar to a favorite",
                "📂 Browse movies by a specific category",
                "❌ Exit Program"
            ]
        ).ask()

        # Handle the user's menu choice
        if action == "🔍 Find movies similar to a favorite":
            movie_input = questionary.text("Enter a movie name (typos are okay!):").ask()
            if movie_input:
                recommender.find_similar_movies(movie_input)

        elif action == "📂 Browse movies by a specific category":
            category = questionary.select(
                "Which category do you want to search?",
                choices=["Genre", "Director", "Language", "Year", "Cast"]
            ).ask()

            if category:
                search_term = questionary.text(f"Enter the {category} you are looking for:").ask()
                if search_term:
                    recommender.browse_by_category(category, search_term)

        elif action == "❌ Exit Program" or action is None:
            print("\nThanks for using the AI Movie Recommender. Goodbye! 👋\n")
            break

        # Add a pause before clearing the screen/looping back
        input("\nPress Enter to return to the main menu...")
```

OUTPUT:-

```
PS C:\Users\HP\OneDrive\Desktop\movie recommendation> python -u "c:\Users\HP\OneDrive\Desktop\movie recommendation\ai_movie_recommendation.py"
? 🍿 Welcome to the AI Movie Recommender! What would you like to do? 📂 Browse movies by a specific category
? Which category do you want to search? Year
? Enter the Year you are looking for: 2000

📂 Top results for '2000' in Year:
--------------------------------------------
1. Hera Pheri (Year: 2000.0 | Dir: Priyadarshan)
2. Gladiator (Year: 2000.0 | Dir: Ridley Scott)
3. Memento (Year: 2000.0 | Dir: Christopher Nolan)
4. Memento (Year: 2000.0 | Dir: Christopher Nolan)
--------------------------------------------
```

```
Press Enter to return to the main menu...
? 🍿 Welcome to the AI Movie Recommender! What would you like to do? 🔍 Find movies similar to a favorite
? Enter a movie name (typos are okay!): Avengers Endgame

(AI Auto-Corrected 'Avengers Endgame' to 'Avengers: Endgame')

🎬 If you liked 'Avengers: Endgame', you should check out:
--------------------------------------------
1. Poor Things (Comedy, Drama, Romance)
2. Goodfellas (Biography, Crime, Drama)
3. Guardians of the Galaxy Vol. 3 (Action, Adventure, Comedy)
4. RRR (Action, Drama)
5. Star Wars: Episode IV - A New Hope (Action, Adventure, Fantasy)
--------------------------------------------

Press Enter to return to the main menu...
```

THANK YOU !!